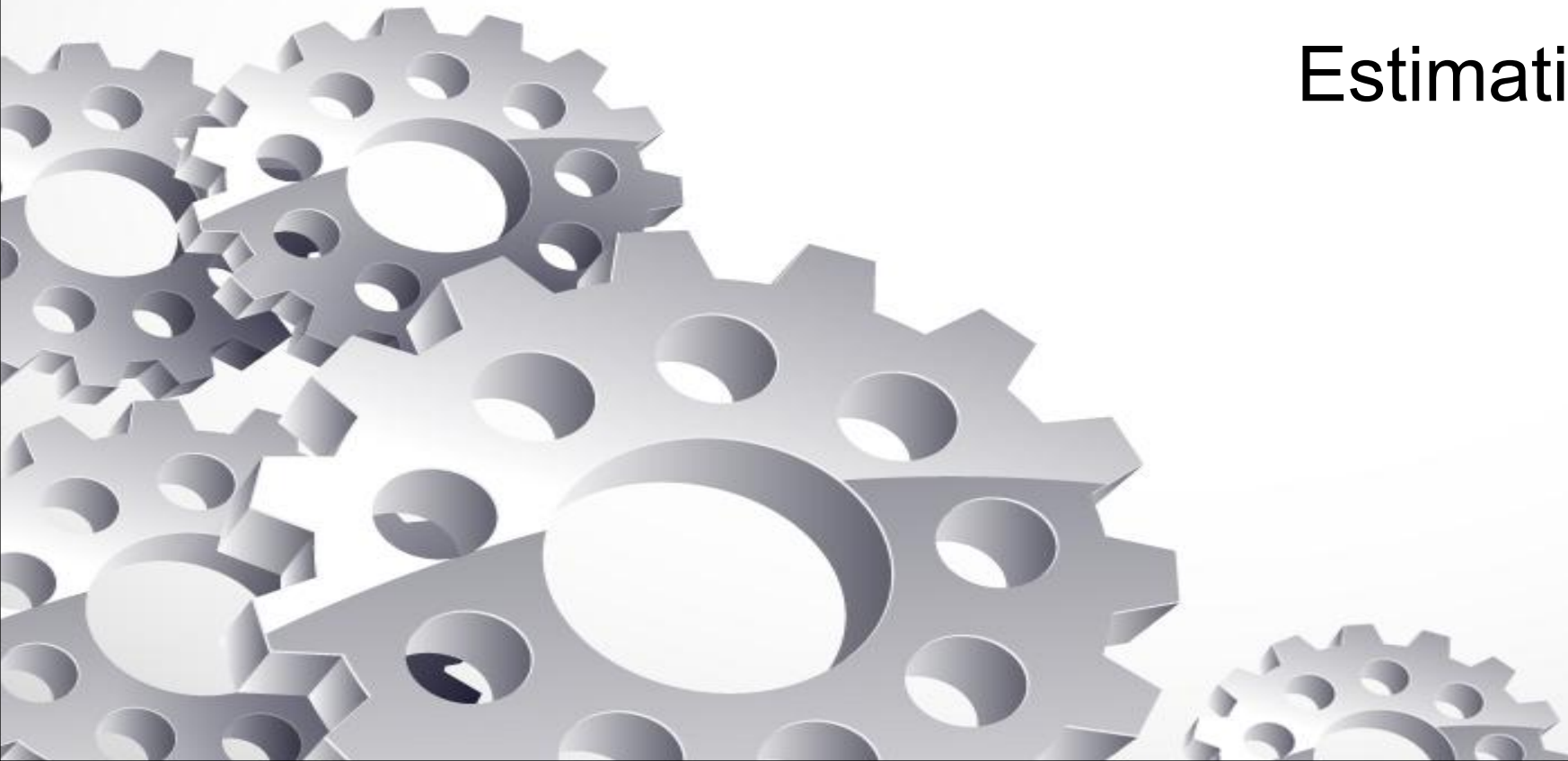


Planning and Product Management

Estimations and metrics



Agile Product Management:



- Communication
- Planning
- Estimation
- Managing the Agile approach by Monitoring progress, targeting and motivating the team
- Managing business involvement and any escalating issues
- Quality and Risk Metrics and Measurements.

Agile Metrics



- Measuring progress is essential in Agile project management, as metrics provide objective insight into team performance and areas for improvement.
- Traditional plan-driven metrics are often inadequate, so Agile relies on leading indicators that reflect team behavior, velocity, quality, and value delivery. These metrics help validate Agile practices, forecast progress, optimize planning, monitor quality, and measure customer value.
- Effective agile metrics share some key traits:
 - ✓ Lightweight and Actionable
 - ✓ Customer-focused
 - ✓ Trend-based
 - ✓ Simple to measure
 - ✓ Automated tracking and Real-time visibility

Common Agile Metrics



- Team-level Metrics**
- Sprint
 - Burndown
 - Velocity
 - Cycle Time
 - Throughput
 - Escape Rate
 - Team Happiness / Morale

- Product-level Metrics**
- Time-to-Market
 - Release
 - Frequency
 - Defects
 - Technical Debt
 - Customer Satisfaction

- Process-level Metrics**
- Value Delivered
 - Return on Investment (ROI)
 - Cost per Iteration
 - Lead Time
 - Flow Efficiency

1.Team-level Metrics



- **Sprint Burndown** - Tracks completion of sprint backlog items over the sprint timeline. Useful for projecting delivery.
- **Velocity** - Number of backlog items (story points) a team completes per sprint on average. Helps estimate workload capacity.
- **Cycle time** - The time taken from starting work on a backlog item to completion. Faster cycle time increases agility.
- **Throughput** - Total number of backlog items completed by a team in a given period (sprint, release, etc). Increased throughput drives delivery.
- **Escape rate** - Percentage of committed backlog items not completed in a sprint. Lower escape rates improve predictability.
- **Happiness/Morale** - Subjective but important metric on team engagement and cohesiveness. Gather feedback frequently.

2. Product-level Metrics



- **Time-to-market** - The elapsed time from initiating a new product or feature until it is released and adopted by customers. Faster time-to-market provides a competitive advantage.
- **Release frequency** - How often new versions of a product are released to customers. Shorter release cycles get value to customers faster through incremental deliveries.
- **Defects** - Count of defects reported on release versions. Defects impact user experience. Target zero production defects.
- **Technical debt** - Accumulated backlog of non-functional work needed to keep a product maintainable. Too much debt slows progress.
- **Customer satisfaction** - Subjective but essential rating of how happy customers are with a product. Gather continuous customer feedback.

3. Process-level Metrics



- **Value delivered** - An estimate of the overall business value provided by agile initiatives through product capabilities released. A higher value increases ROI.
- **Return on investment (ROI)** - Measures net benefit of agile projects by comparing value delivered to cost. Helps demonstrate success.
- **Cost per iteration** - The summed cost of resources, tools, and overhead needed to complete an iteration. Lower is better for scaling agile.
- **Lead time** - The total elapsed time for a backlog item to go from request to release. Faster lead time increases responsiveness.
- **Flow efficiency** - Assesses how smoothly work moves through agile process stages without stoppage or delay. Improving flow reduces waste.

Estimation as the Basis for Agile Metrics

The Sydney Opera House was expected to complete in 4 years with a budget of AUD 7 million. It took 14 years to complete and cost AUD 102 million. In short, the project experienced a delay of ten years and went 1457% over budget.

The Sydney opera house example shows how quickly projects can get off track. If you don't conduct accurate estimations, their product will get delayed, and the competition will gain an advantage.

That's why Agile estimation is essential. It can help you evaluate the time and effort you need to complete work items in your product backlog. It will further enhance sprint planning.

However, **it's challenging to predict accurate estimates. How would a company know the amount of time it will take to complete a product backlog item in advance? How can they account for unforeseen impediments that arise?**

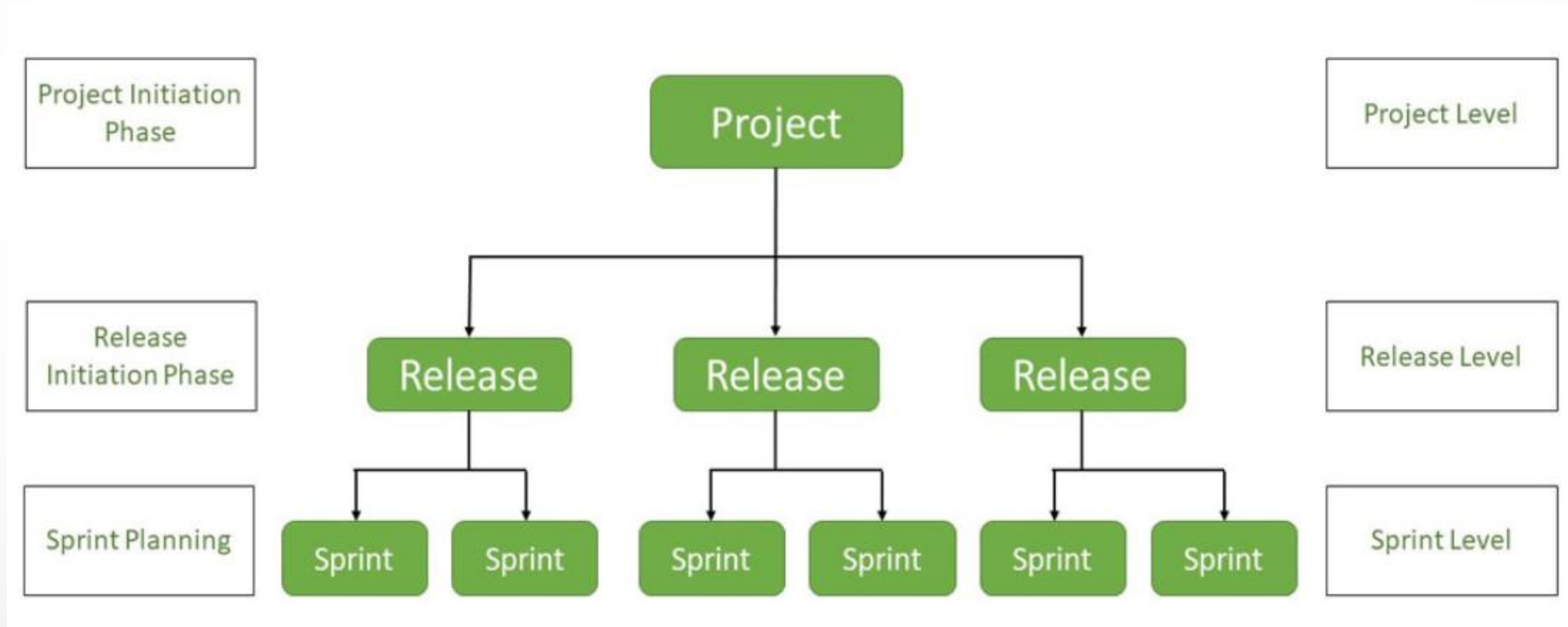
That is where Agile estimation techniques come to the rescue. They keep your project on track by helping you estimate your **project's time, budget, and effort**.

A rule of thumb for making accurate estimations is to put effort into strategizing around the proper Agile estimation techniques. Therefore, understanding agile story estimation techniques should be a top priority for a business looking to develop a new product.



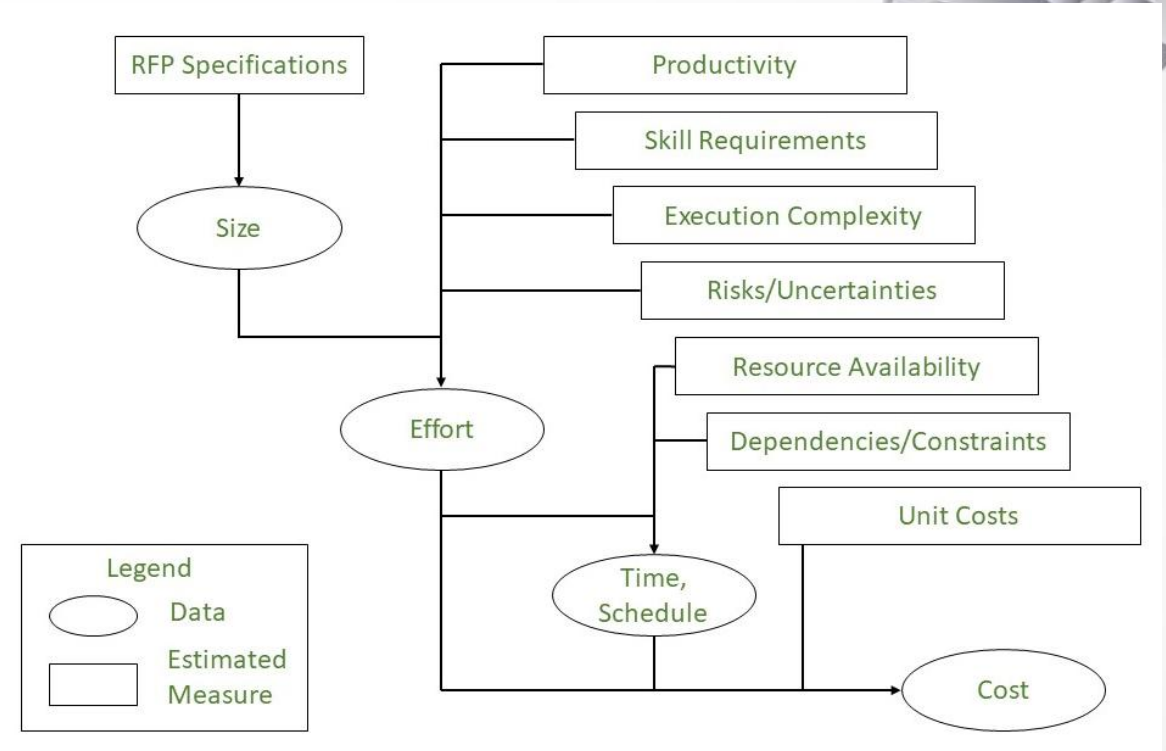
Agile Planning and Estimation

- Planning and Estimation in Agile projects are generally done at three different stages, which are:
 - ✓ Project Initiation Level
 - ✓ Release Level
 - ✓ Sprint Level (i.e., an iteration within each Release)



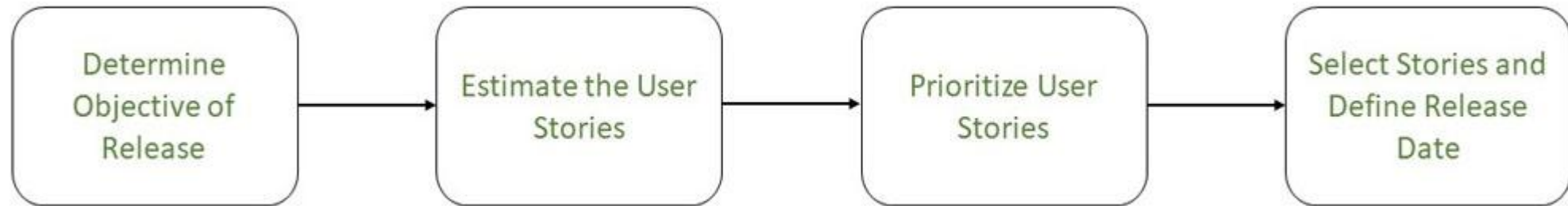
Project Level : Planning and estimation

- Project-level planning provides a **big-picture view** of the Agile project, including vision, roadmap, value delivery, risk coverage, and release strategy. The Product Owner plays a central role in defining goals, creating and prioritizing the Product Backlog, and guiding the team on strategic direction. This level of planning ensures predictability, stakeholder alignment, and risk mitigation through activities like backlog creation, release planning, workshops, and the use of ALM tools. High-level estimation techniques are used to forecast releases, timelines, staffing, and effort, and these estimates are continuously refined.



Release Level : Planning

Release planning translates the product vision into **time-boxed releases** consisting of multiple sprints. It focuses on selecting, estimating, and prioritizing user stories from the Product Backlog to meet release objectives. Estimation at this level is done using **story points** and techniques such as Planning Poker, Wideband Delphi, and Estimation by Analogy. Release planning enables incremental delivery, early feedback, and alignment with business goals while allowing scope adjustments as the project evolves.

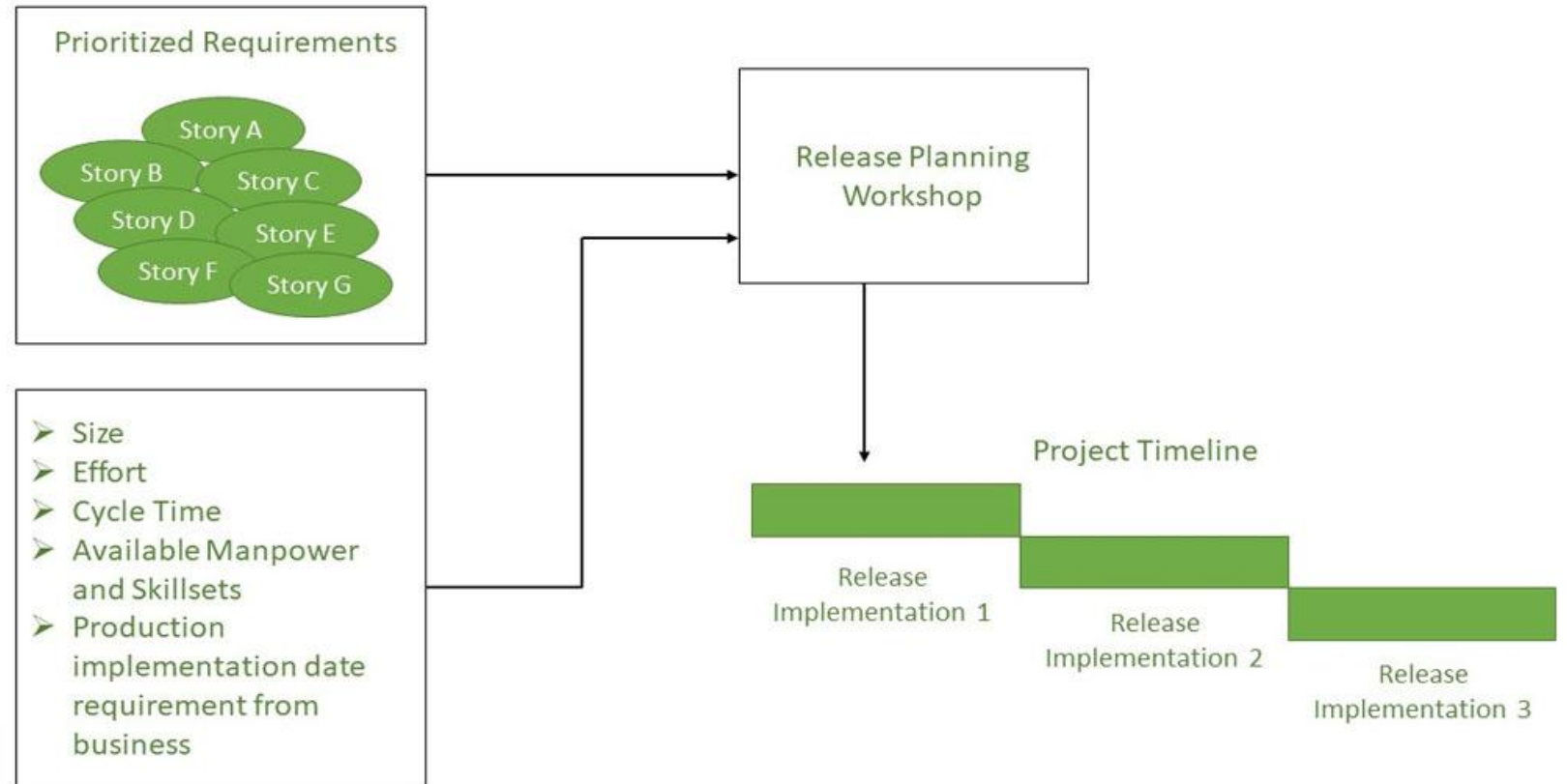


Release Level Estimation

- Estimation is done using **User Stories** from the Product Backlog
- User stories are estimated using **Story Points**
- Inputs considered:
 - Business priority
 - Team capacity
 - Previous velocity (if available)
- Helps decide:
 - Total story points per release
 - Number of releases required

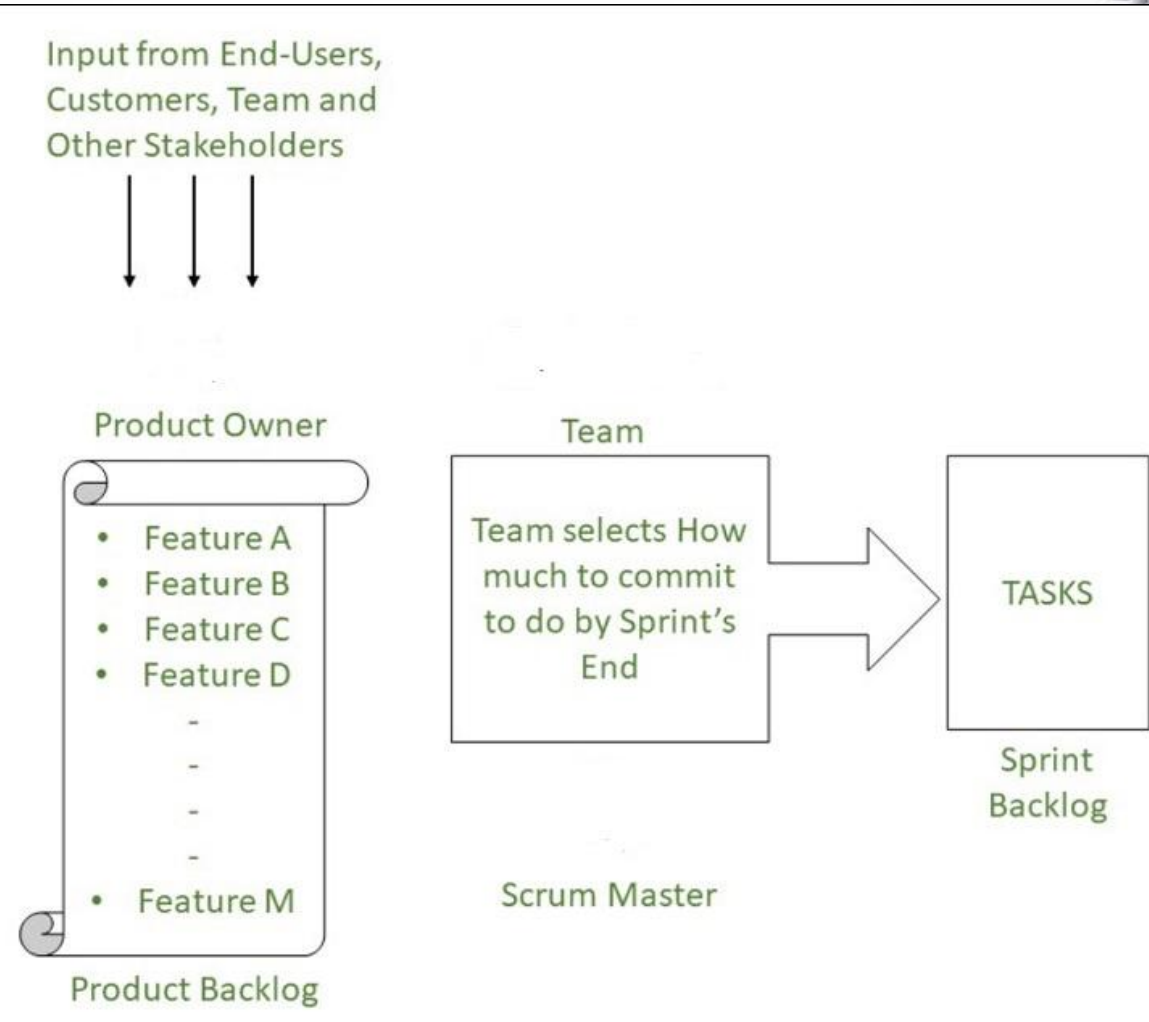
Estimation Techniques Used:

- **Planning Poker (Fibonacci series)**
- **Wideband Delphi (consensus-based)**
- **Estimation by Analogy**



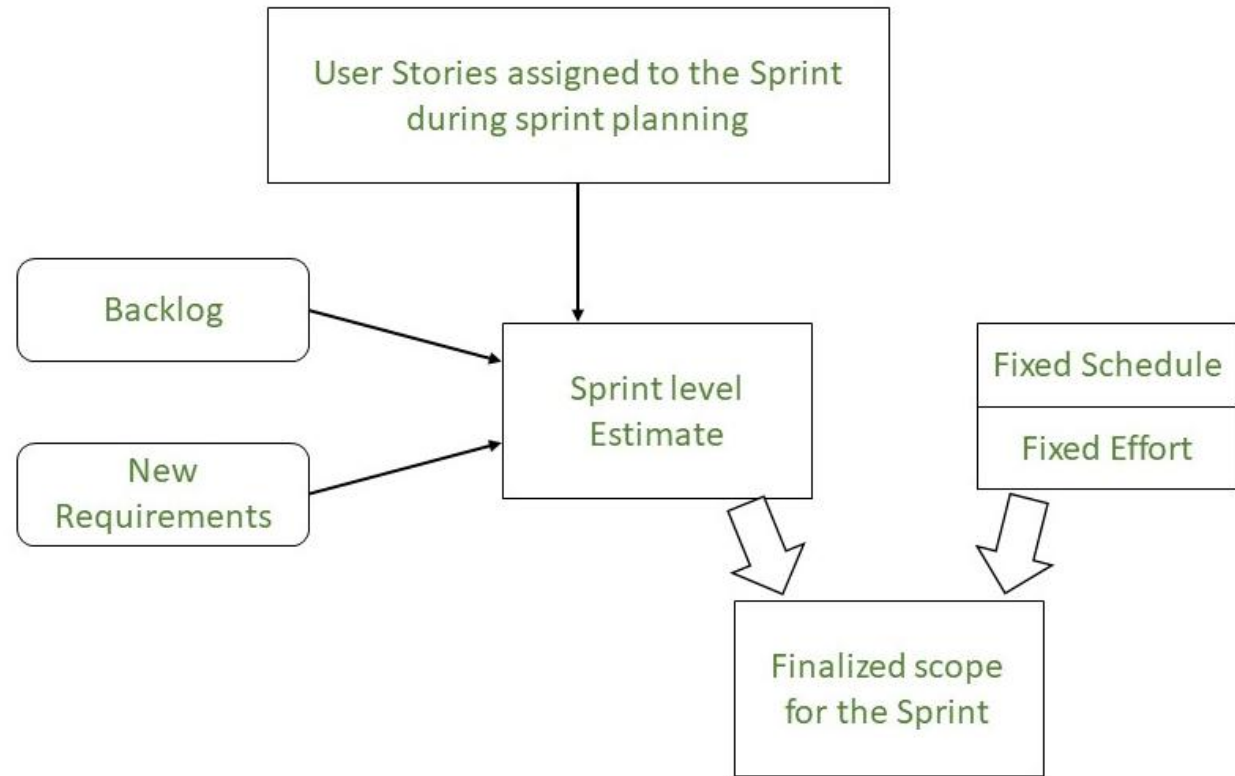
Sprint-Level : Planning

- Conducted at the start of each Sprint
- Defines:
 - Sprint goal
 - User stories selected for the sprint
- Team decides what can be delivered based on:
 - Team capacity
 - Velocity
- User stories are broken down into tasks, forming the Sprint Backlog
- Involves Product Owner, Scrum Master, and entire Scrum Team



Sprint-Level: Estimation

- Uses bottom-up estimation
- Tasks are estimated in hours or days
- Estimation is done by the task owner
- Based on the Definition of Done (DoD)
- Remaining task effort is updated daily and used to generate Sprint Burndown Charts
- Ensures realistic sprint commitment and better tracking



Velocity

$$\text{Velocity} = \frac{\text{Total Story Points Completed}}{\text{Number of Sprints}}$$



✂ Elements

- **Total Story Points Completed:** Sum of story points finished
- **Number of Sprints:** Sprints considered for calculation

✓ Solved Example

Total = 18 + 22 + 20 = 60 Sprints = 3

$$\text{Velocity} = \frac{60}{3} = 20 \text{ story points per sprint}$$

🔪 Practice Question

A team completes 15, 25, 30, and 20 story points in four sprints. Find the average velocity.

Sprint Burndown (Remaining Work)

$$\text{Remaining Work} = \text{Total Planned Work} - \text{Completed Work}$$

⚙️ Elements

- **Total Planned Work:** Story points committed
- **Completed Work:** Story points finished

✓ Solved Example

A sprint starts with 50 story points. After one week, 32 points are completed.

Find the **remaining work**.

$$50 - 32 = 18 \text{ story points}$$

🔪 Practice Question

A sprint starts with 40 story points. 27 are completed. Calculate the remaining work.

Escape Rate

$$\text{Escape Rate} = \frac{\text{Incomplete Committed Items}}{\text{Total Committed Items}} \times 100$$

✂ Elements

- **Incomplete Committed Items:** Planned but not completed
- **Total Committed Items:** Initially committed items
- ✓ **Solved Example** - A team commits to 16 user stories but completes only 12.
Calculate the **escape rate**.

$$\text{Incomplete} = 16 - 12 = 4$$

$$\frac{4}{16} \times 100 = 25\%$$

📌 Practice Question

A team commits to 20 stories but completes 17. Find the escape rate.

Cycle Time

$$\text{Cycle Time} = \text{Completion Date} - \text{Start Date}$$

⚙️ Elements

- **Start Date:** When work begins
- **Completion Date:** When work ends

✓ **Solved Example** - A task starts on Day 3 and finishes on Day 9. Find the **cycle time**.

$$9 - 3 = 6 \text{ days}$$

🔪 Practice Question

A task starts on Day 5 and finishes on Day 11. Calculate the cycle time.

Throughput

Throughput = Number of Items Completed per Time Period

✂ Elements

- **Items Completed:** Stories/tasks finished
- **Time Period:** Sprint/week

✓ **Solved Example** - A team completes 14 user stories in a sprint. Calculate the throughput.

Throughput = 14 *stories per sprint*

📌 Practice Question

A team completes 18 backlog items in a sprint. Find the throughput.

Time-to-Market


$$\text{Time-to-Market} = \text{Release Date} - \text{Start Date}$$

⚙️ Elements

- **Start Date:** Beginning of development
- **Release Date:** Feature release date

✓ **Solved Example** - Development started on 1st February and release happened on 2nd April.
Calculate the **time-to-market**.

Time-to-Market = 60 days

📌 Practice Question

Development starts on 10th January and release is on 25th February. Find the time-to-market.

Defect Density

$$\text{Defect Density} = \frac{\text{Number of Defects}}{\text{Size of Software}}$$



✎ Elements

- **Number of Defects:** Bugs found
- **Size of Software:** Modules / KLOC / FP

✓ **Solved Example** - A software system has 12 defects across 4 modules.
Calculate the **defect density**

$$\frac{12}{4} = 3 \text{ defects per module}$$

✎ Practice Question

A system has 20 defects in 5 modules. Calculate defect density.

Value Delivered

$$\text{Value Delivered} = \sum \text{Business Value of Completed Stories}$$

⚙️ Elements

- **Business Value:** Assigned value to each story
- **Completed Stories:** Finished work items

✓ **Solved Example** - A sprint completes stories with business values 15, 25, and 10.
Calculate the **value delivered**.

$$15 + 25 + 10 = 50$$

🔪 Practice Question

Stories with values 20, 30, and 40 are completed. Find the value delivered.

Return on Investment (ROI)

$$\text{ROI} = \frac{\text{Value Delivered} - \text{Cost}}{\text{Cost}} \times 100$$

✎ Elements

- **Value Delivered:** Business benefit
- **Cost:** Total project cost

✓ **Solved Example** - A project costs ₹1,50,000 and delivers value worth ₹2,00,000. Calculate the **ROI**.

$$\frac{2,00,000 - 1,50,000}{1,50,000} \times 100 = 33.33\%$$

✎ Practice Question

A project costs ₹1,00,000 and delivers value ₹1,40,000. Calculate ROI.

Lead Time

$$\text{Lead Time} = \text{Delivery Date} - \text{Request Date}$$



✎ Elements

- **Request Date:** When work is requested
- **Delivery Date:** When delivered

✓ **Solved Example** - A feature request is raised on Day 4 and delivered on Day 16.
Calculate the **lead time**

$$16 - 4 = 12 \text{ days}$$

📌 Practice Question

A request is made on Day 7 and delivered on Day 21. Find the lead time.